Séance 4

Préparer FoodFast au monde réel

Concurrence · JDBC · le bug et la faille

Objectifs de la séance

AA7 : identifier et corriger une **race condition** et une **injection SQL**.

1. **Concurrence** (≈ 30 min) : lancer des tâches en parallèle, voir des données se perdre, corriger
2. 🔒 **TOCTOU** : un code promo utilisé 200 fois
3. **JDBC** (≈ 30 min) : lire et écrire des commandes dans une base H2
4. 🔒 **Injection SQL** : vider la base clients avec une apostrophe

Vendredi, 19 h 55

Des centaines de clients valident leur panier **dans la même seconde**.

Pour chaque commande, ≈ 300 ms d'**attente** : restaurant, base, e-mail.

Traitement	1 000 commandes
Une par une	plus de **5 minutes**
En parallèle	moins d'**une seconde**

> Plusieurs threads, c'est plus rapide. Mais ils **partagent la mémoire**.

Partie 1

Concurrence : le bug et la faille

Lancer des tâches : `ExecutorService`

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 1; i <= 10_000; i++) {
        int n = i;
        executor.submit(() -> notifyRestaurant(n)); // 100 ms chacune
    }
} // close() ATTEND la fin de toutes les tâches

IO.println("10 000 restaurants prévenus");     // en ≈ 0,2 s
```

- Un `ExecutorService` gère les threads à votre place
- **try-with-resources** (S3) : plus de `shutdown()` oublié
- `submit` renvoie un `Future` : `get()` donne le résultat ou l'erreur

📖 Support §1.2 — ExecutorService

Les threads virtuels, en une slide

	Thread plateforme	Thread virtuel (Java 21)
Géré par	le système	la JVM
Coût	≈ 1 Mo	quelques centaines d'octets
Combien ?	quelques milliers	des **millions**
Pour	calcul intensif	tâches qui **attendent** (réseau, base)

- Pendant une attente, la JVM « décroche » le thread virtuel
- On ne les met **pas en pool** : un par tâche
- On limite la ressource rare (connexions à la base), pas les threads

🤔 Prédisez la sortie

```
class OrderCounter {
    private int count = 0;
    void increment() { count++; }
    int get() { return count; }
}
var counter = new OrderCounter();
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int t = 0; t < 1_000; t++) {
        executor.submit(() -> {
            for (int i = 0; i < 1_000; i++) counter.increment();
        });
    }
}
IO.println(counter.get()); // 1 000 tâches × 1 000 incréments = ?
```

🟢 Démo : les commandes disparaissent

```
Commandes comptées : 174491 (attendu : 1000000)
Commandes comptées : 247164 (attendu : 1000000)
Commandes comptées : 152176 (attendu : 1000000)
```

`count++`, c'est **trois** étapes : lire, ajouter 1, écrire.

```
Thread A                    Thread B
lit count (41)
                            lit count (41)
écrit 42
                            écrit 42   ← l'incrément de A est perdu
```

📖 Support §1.4 — Race condition

🟢 Correction 1 : `AtomicInteger`

```
class OrderCounter {
    private final AtomicInteger count = new AtomicInteger();

    void increment() {
        count.incrementAndGet(); // lire + ajouter + écrire : indivisible
    }

    int get() {
        return count.get();
    }
}
```

```
Commandes comptées : 1000000 (attendu : 1000000)
```

> Une **seule** variable partagée → un type atomique.

📖 Support §1.5 — AtomicInteger

🟢 Correction 2 : `synchronized`

```
class SalesStats {
    private int orderCount = 0;
    private BigDecimal revenue = BigDecimal.ZERO;

    synchronized void record(BigDecimal orderTotal) {
        orderCount++;                          // les deux champs
        revenue = revenue.add(orderTotal);     // changent ensemble
    }

    synchronized String summary() {            // la lecture aussi !
        return orderCount + " commandes, " + revenue + " €";
    }
}
```

📖 Support §1.6 — synchronized

Quel outil pour quel besoin ?

Besoin	Outil
Un compteur, une variable	`AtomicInteger`, `AtomicLong`
Plusieurs variables cohérentes	`synchronized`
Une `Map` partagée	`ConcurrentHashMap`
Un objet qui ne change jamais	**rien** : `record`, `String`, `BigDecimal`

> La meilleure protection contre les race conditions : l'**immuabilité**.

📖 Support §1.6 — tableau récapitulatif

`ConcurrentHashMap` : thread-safe… appel par appel

`DeliveryPlatform` (S2) stocke ses commandes dans une `HashMap` : **pas** faite pour plusieurs threads.

```
// S4 : la plateforme devient thread-safe
private final Map<String, Order> orders = new ConcurrentHashMap<>();

public void placeOrder(Order order) {
    if (orders.putIfAbsent(order.getId(), order) != null) {
        throw new IllegalStateException("Commande déjà enregistrée");
    }
}
```

📖 Support §1.7 — DeliveryPlatform thread-safe

🤔 Prédisez la sortie

```
Map<Dish, Integer> unsafe = new ConcurrentHashMap<>();
Map<Dish, Integer> safe = new ConcurrentHashMap<>();
// 1 000 tâches virtuelles, chacune exécute 100 fois :
int current = unsafe.getOrDefault(pizza, 0);
unsafe.put(pizza, current + 1);

safe.merge(pizza, 1, Integer::sum);
// ...
IO.println(unsafe.get(pizza) + " / " + safe.get(pizza));
```

Les deux maps sont des `ConcurrentHashMap`. Même résultat ?

📖 Support §1.7 — DishPopularity.java

Les opérations atomiques de `ConcurrentHashMap`

Méthode	Atomiquement…	FoodFast
`merge(k, 1, Integer::sum)`	ajoute ou crée	compter les plats
`putIfAbsent(k, v)`	insère si absent	enregistrer une commande
`compute(k, f)`	recalcule la valeur	décrémenter un stock
`remove(k)`	supprime et **renvoie**	consommer un code promo

> Une suite `get` → `if` → `put` sur une donnée partagée est **suspecte**.

📖 Support §1.7 — tableau complet

🔒 TOCTOU : « vérifier puis agir »

Time **O**f **C**heck to **T**ime **O**f **U**se (CWE-367)

1. Le code **vérifie** une condition : « ce code promo est-il valide ? »
2. Un autre thread change la situation
3. Le code **agit** comme si la condition était encore vraie

Une race condition **exploitable** : un attaquant envoie 200 requêtes identiques en même temps.

📖 Support §1.8 — TOCTOU

🔒 Le code promo `BIENVENUE10` (usage unique)

```
private final Map<String, BigDecimal> unusedCodes =
        new ConcurrentHashMap<>();

// ❌ TOCTOU
BigDecimal redeem(String code) throws InterruptedException {
    BigDecimal amount = unusedCodes.get(code);  // CHECK
    if (amount == null) throw new InvalidPromoCodeException();
    saveDiscount(code, amount);                 // ≈ 5 ms en base
    unusedCodes.remove(code);                   // USE
    return amount;
}
```

📖 Support §1.8 — PromoCodeRace.java

🔓 Démo : 200 requêtes simultanées

```
Code BIENVENUE10 accepté 200 fois, refusé 0 fois
```

2 000 € de remise au lieu de 10 €.

- Chaque appel à la map est thread-safe…
- … mais la **séquence** `get` → `remove` ne l'est pas
- Les 200 threads passent le `get` avant le premier `remove`

📖 Support §1.8 — PromoCodeRace.java

🔒 La correction : vérifier ET agir en une étape

```
// ✅ remove renvoie la valeur ET supprime l'entrée, atomiquement
BigDecimal redeem(String code) throws InterruptedException {
    BigDecimal amount = unusedCodes.remove(code); // CHECK + USE
    if (amount == null) throw new InvalidPromoCodeException();
    saveDiscount(code, amount);    // seul le gagnant arrive ici
    return amount;
}
```

```
remove atomique : accepté 1 fois sur 200
```

📖 Support §1.8 — PromoCodeSafe.java

🔒 Et avec un verrou ?

❌ Ne protège pas

```
synchronized void deleteAll(
        User user) {
    if (user.isAdmin()) {
        orders.clear();
    }
}
// un autre thread retire les
// droits SANS prendre ce verrou
```

✅ Protège

```
synchronized BigDecimal redeem(
        String code) {
    // CHECK… et USE ici
}
synchronized void addCode(...)
// TOUS les accès : CE verrou
```

📖 Support §1.8 — Pourquoi l'ancienne correction était fausse

Concurrence : ce qu'il faut retenir

- `ExecutorService` en **try-with-resources**, threads **virtuels** pour les attentes
- `count++` n'est pas atomique ; un test qui passe ne prouve rien
- `AtomicInteger`, `synchronized` (**même verrou pour lire et écrire**)
- `ConcurrentHashMap` : `merge`, `compute`, `putIfAbsent`, `remove`
- 🔒 TOCTOU : vérification et action en **une seule** opération atomique

> Repérez les `if` qui testent un état partagé avant de le modifier.

📖 Support §1.9 à §1.13 — stock, deadlocks, fichiers, scoped values

Partie 2

JDBC : le bug et la faille

🟢 JDBC et H2

Une `Map` en mémoire : **un redémarrage, et tout est perdu**.

```
Votre code ──► API JDBC (java.sql)
                      ▼
               Driver (.jar : H2, PostgreSQL…)
                      ▼
               Base de données
```

H2 **en mémoire** : zéro installation, une dépendance Maven.

```
jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1
```

Se connecter : trois objets, trois `close()`

```
String url = "jdbc:h2:mem:foodfast;DB_CLOSE_DELAY=-1";
String user = System.getenv().getOrDefault("FOODFAST_DB_USER", "sa");
String password =
        System.getenv().getOrDefault("FOODFAST_DB_PASSWORD", "");

try (Connection conn = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = conn.prepareStatement("SELECT H2VERSION()");
     ResultSet rs = ps.executeQuery()) {
    rs.next();
    IO.println("Connecté à H2 " + rs.getString(1));
} // rs, ps, puis conn sont fermés
```

📖 Support §2.2 — Connection et try-with-resources

🟢 Le schéma : SQL lisible avec un text block

```
private static final String CREATE_TABLE = """
        CREATE TABLE IF NOT EXISTS orders (
            id            VARCHAR(64)   PRIMARY KEY,
            customer_name VARCHAR(100)  NOT NULL,
            total         DECIMAL(10,2) NOT NULL,
            status        VARCHAR(20)   NOT NULL,
            created_at    TIMESTAMP     NOT NULL
        )
        """;
```

- `BigDecimal` ↔ `DECIMAL`, jamais `DOUBLE` (S2)
- `enum` ↔ `VARCHAR` (son `name()`), `LocalDateTime` ↔ `TIMESTAMP`

📖 Support §2.4 — Text blocks et types

🟢 Écrire : `PreparedStatement`

```
private static final String INSERT = """
        INSERT INTO orders (id, customer_name, total, status, created_at)
        VALUES (?, ?, ?, ?, ?)
        """;
void save(Order order) throws SQLException {
    try (Connection conn = connect();
         PreparedStatement ps = conn.prepareStatement(INSERT)) {
        ps.setString(1, order.getId());           // positions dès 1
        ps.setString(2, order.getCustomer().getName());
        ps.setBigDecimal(3, order.getTotal());
        ps.setString(4, order.getStatus().name());
        ps.setObject(5, order.getCreatedAt());    // LocalDateTime
        ps.executeUpdate();
    } // ...
```

📖 Support §2.5 — PreparedStatement

Lire : `ResultSet` → objet

```
Optional<OrderSummary> findById(String id) throws SQLException {
    try (Connection conn = connect();
         PreparedStatement ps = conn.prepareStatement(SELECT_BY_ID)) {
        ps.setString(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) return Optional.empty(); // aucune ligne
            return Optional.of(new OrderSummary(
                    rs.getString("id"),
                    rs.getString("customer_name"),
                    rs.getBigDecimal("total"),
                    OrderStatus.valueOf(rs.getString("status")),
                    rs.getObject("created_at", LocalDateTime.class)));
        }
    } // ...
```

📖 Support §2.6 — OrderRepository complet

🔓 Injection SQL : la recherche par client

```
// ❌ La saisie de l'utilisateur est collée dans le SQL
List<String> findByCustomerUnsafe(Connection conn, String name)
        throws SQLException {
    String sql = "SELECT id, customer_name, total FROM orders"
            + " WHERE customer_name = '" + name + "'";
    try (Statement st = conn.createStatement();
         ResultSet rs = st.executeQuery(sql)) {
        return readRows(rs);
    }
}
```

Recherche « Alice » → `[order-1 (Alice, 31.90 €)]`

📖 Support §2.7 — SqlInjectionDemo.java

🤔 Prédisez la sortie

L'utilisateur saisit comme nom de client :

```
' OR '1'='1
```

```
findByCustomerUnsafe(conn, "' OR '1'='1");
```

La table contient les commandes d'Alice, Bob et Chloé.

Que renvoie la recherche ?

🔓 Démo : toutes les commandes de tous les clients

```
SQL exécuté : SELECT id, customer_name, total FROM orders
              WHERE customer_name = '' OR '1'='1'
[order-1 (Alice, 31.90 €), order-2 (Bob, 18.00 €),
 order-3 (Chloé, 42.50 €)]
```

- La première `'` **ferme** la chaîne SQL
- `OR '1'='1'` devient du **code** : vrai pour toutes les lignes
- Variantes : `admin' --` (contourne un login), `UNION SELECT` (lit d'autres tables)

📖 Support §2.7 — Injection SQL

🔒 La correction : `PreparedStatement`

```
// ✅ Structure figée ; la saisie n'est qu'une valeur
String sql =
    "SELECT id, customer_name, total FROM orders WHERE customer_name = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, name);
    try (ResultSet rs = ps.executeQuery()) {
        return readRows(rs);
    }
}
```

Même saisie `' OR '1'='1` → `[]` (aucun client ne s'appelle ainsi).

> La requête est analysée **avant** de connaître la valeur : la valeur ne devient **jamais** du SQL.

📖 Support §2.7 — Pourquoi PreparedStatement protège

🔒 Les limites du `?`

Un `?` remplace une **valeur**, jamais un nom de colonne. `ORDER BY ?` ne trie pas.

```
// ✅ Liste blanche : la saisie CHOISIT, elle n'est jamais copiée
String sortColumn(String userChoice) {
    return switch (userChoice) {
        case "date" -> "created_at";
        case "total" -> "total";
        default -> throw new IllegalArgumentException("Tri non autorisé");
    };
}
String sql = "SELECT id FROM orders ORDER BY " + sortColumn(choice);
```

📖 Support §2.8 — Liste blanche

🔒 Les secrets hors du code

❌ Codé en dur

```
String password = "secret";
```

- finit dans Git, pour toujours
- des robots scannent GitHub
- fuite → **changer** le secret

✅ Lu dans l'environnement

```
String password = System
    .getenv("FOODFAST_DB_PASSWORD");
```

- `.env` dans `.gitignore`
- gestionnaire de secrets en prod
- en prod : **pas** de valeur par défaut

📖 Support §2.3 — Identifiants hors du code

🔒 Le moindre privilège

```
// Au déploiement, compte administrateur :
st.execute("GRANT SELECT, INSERT ON orders TO foodfast_app");

// L'application se connecte avec foodfast_app :
st.execute("DROP TABLE orders");
```

```
INSERT autorisé
DROP TABLE refusé (code 90096)
```

> Si une injection passe quand même, les dégâts sont limités aux droits du compte.

📖 Support §2.3 — LeastPrivilege.java

Ce qu'il faut retenir

- 🟢 `ExecutorService` en try-with-resources ; threads virtuels pour les attentes
- 🟢 Race condition → `AtomicInteger`, `synchronized`, `ConcurrentHashMap.merge`
- 🔒 TOCTOU → vérifier **et** agir en une seule opération atomique
- 🟢 JDBC : `Connection`, `PreparedStatement`, `ResultSet` en try-with-resources
- 🔒 Injection SQL → **toujours** des `?` ; liste blanche pour les colonnes
- 🔒 Secrets dans l'environnement ; compte au moindre privilège

Pour aller plus loin (support)

§	Sujet	Niveau
1.9 – 1.11	Stock et `compute`, deadlocks, TOCTOU fichiers	🔒 🔵
1.12	Scoped values (Java 25)	🟣
2.9	Transactions : `commit` / `rollback`	🔵
3.1	Hacher des mots de passe : PBKDF2	🔵 🔒
3.2	API KDF (Java 25) : pourquoi HKDF ≠ mot de passe	🟣 🔒
3.3	Désérialisation Java : chaînes de gadgets	🔒

📖 Support §1.9 à §3.3